Actividad práctica — Orquestación básica con Docker Compose

Introducción

En esta actividad se configuró un entorno multicontenedor utilizando **Docker Compose**, permitiendo ejecutar varios servicios de forma simultánea dentro de un mismo sistema.
Se trabajó con dos servicios principales:

- un servidor web usando la imagen **Nginx**
- una base de datos usando la imagen **PostgreSQL**

El objetivo fue aprender a definir múltiples contenedores en un solo archivo docker-compose.yml, ejecutar los servicios y verificar su funcionamiento.

Objetivo de la actividad

Definir y ejecutar un entorno con múltiples servicios usando Docker Compose, permitiendo la comunicación entre un servidor web y una base de datos dentro de un mismo entorno controlado.

Paso 1 — Crear el directorio del proyecto

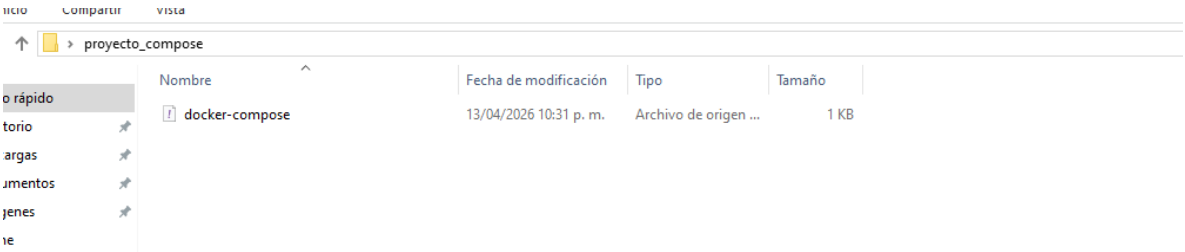
Se creó una carpeta llamada:

proyecto_compose

Dentro de esta carpeta se creó el archivo principal:

docker-compose.yml

Esto permitió organizar los servicios dentro de un entorno único.



Paso 2 — Crear el archivo docker-compose.yml

Se definió el archivo docker-compose.yml, el cual permite configurar múltiples contenedores dentro de un solo entorno.

El archivo incluye:

- servicio web con Nginx
- servicio de base de datos PostgreSQL
- volumen para persistencia de datos
- mapeo de puertos

Archivo docker-compose.yml utilizado

```
version: '3.8'

services:

  web:
    image: nginx
    container_name: web-nginx
    ports:
      - "8080:80"

  db:
    image: postgres:15
    container_name: db-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: admin1234
      POSTGRES_DB: basedatos
    ports:
```

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

volumes:

postgres_data:

Este archivo define los servicios necesarios para ejecutar la aplicación multicontenedor solicitada en la actividad.

Bienvenido | docker-compose.yml

docker-compose.yml

```yaml
version: '3.8'

services:

  web:
    image: nginx
    container_name: web-nginx
    ports:
      - "8080:80"

  db:
    image: postgres:15
    container_name: db-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: yeison
      POSTGRES_DB: basedatos
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Paso 3 — Levantar los servicios

Una vez creado el archivo de configuración, se ejecutó el siguiente comando en la terminal:

docker compose up -d

Este comando permitió iniciar todos los servicios definidos en el archivo docker-compose.yml.

El parámetro:

-d

permite ejecutar los contenedores en segundo plano.

```
PS C:\Users\demoo\Desktop\proyecto_compose> docker compose up -d
time="2026-04-13T22:32:00-05:00" level=warning msg="C:\\Users\\demoo\\Desktop\\proyecto_compose\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pl
ease remove it to avoid potential confusion"
[+] up 29/29
 ✓ Image nginx                          Pulled                                                                                                    10.7s
 ✓ Image postgres:15                    Pulled                                                                                                    42.3s
 ✓ Network proyecto_compose_default     Created
 ✓ Volume proyecto_compose_postgres_data Created
 ✓ Container web-nginx                  Created
 ✓ Container db-postgres                Created
PS C:\Users\demoo\Desktop\proyecto_compose>
```

Paso 4 — Verificar los contenedores

Para verificar que los contenedores estaban ejecutándose correctamente, se utilizó el siguiente comando:

docker compose ps

Este comando muestra el estado de todos los servicios definidos en Docker Compose.

```
PS C:\Users\demoo\Desktop\proyecto_compose> docker compose ps
time="2026-04-13T22:43:54-05:00" level=warning msg="C:\\Users\\demoo\\Desktop\\proyecto_compose\\docker-compose.yml: the attribute `version` i
ease remove it to avoid potential confusion"
NAME          IMAGE          COMMAND                  SERVICE   CREATED          STATUS          PORTS
db-postgres   postgres:15    "docker-entrypoint.s…"   db        11 minutes ago   Up 11 minutes   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
web-nginx     nginx          "/docker-entrypoint.…"   web       11 minutes ago   Up 11 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS C:\Users\demoo\Desktop\proyecto_compose>
```

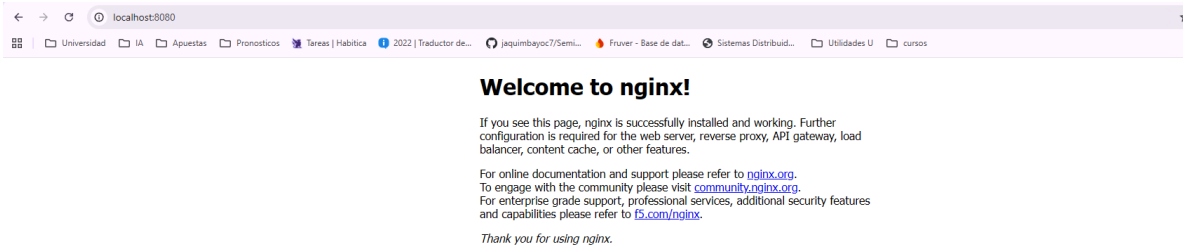
Paso 5 — Verificar el servidor web

Se abrió el navegador web y se ingresó la siguiente dirección:

http://localhost:8080

El navegador mostró la página de bienvenida de **Nginx**, confirmando que el servicio web estaba funcionando correctamente.

La actividad solicita validar que Nginx responda en el puerto 8080.



Paso 6 — Verificar la base de datos

Se verificó que la base de datos PostgreSQL estuviera en ejecución utilizando herramientas como:

- pgAdmin
- Docker logs
- Docker inspect

Esto permitió confirmar que la base de datos fue creada correctamente y que el contenedor estaba activo.

La actividad indica comprobar que la base de datos esté en ejecución.

```
PostgreSQL Database directory appears to contain a database; Skipping initialization

2026-04-14 03:46:10.480 UTC [1] LOG:  starting PostgreSQL 15.17 (Debian 15.17-1.pgdg13+1) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0,
2026-04-14 03:46:10.482 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2026-04-14 03:46:10.482 UTC [1] LOG:  listening on IPv6 address "::", port 5432
2026-04-14 03:46:10.487 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-04-14 03:46:10.499 UTC [29] LOG:  database system was shut down at 2026-04-14 03:45:45 UTC
2026-04-14 03:46:10.515 UTC [1] LOG:  database system is ready to accept connections
PS C:\Users\demoo\Desktop\proyecto compose>
```

Paso 7 — Configurar persistencia de datos

Para garantizar que los datos no se pierdan al reiniciar los contenedores, se configuró un volumen dentro del archivo docker-compose.yml.

El volumen utilizado fue:

volumes:

  postgres_data:

Este volumen permite almacenar los datos de PostgreSQL fuera del contenedor.

La actividad solicita configurar un volumen y validar la persistencia de datos.

Paso 8 — Reiniciar los contenedores

Se ejecutaron los siguientes comandos:

docker compose down

docker compose up -d

Estos comandos permitieron detener y reiniciar los servicios.

Después del reinicio, se verificó que los datos almacenados en la base de datos permanecieran intactos, confirmando que la persistencia funcionaba correctamente.

Paso 9 — Detener los servicios

Al finalizar la actividad, se ejecutó el siguiente comando:

docker compose down

Este comando detiene todos los contenedores definidos en el archivo docker-compose.yml.

La guía recomienda detener los contenedores cuando se termine la práctica.

Entregables solicitados

De acuerdo con la actividad, los entregables incluyen:

- archivo docker-compose.yml funcional
- capturas de docker compose ps
- captura de Nginx en el navegador
- evidencia de la base de datos creada
- informe breve con explicación

Estos requisitos corresponden directamente a lo solicitado en la guía de la actividad.

Reflexión — Ventajas de usar Docker Compose

El uso de Docker Compose ofrece múltiples ventajas en el desarrollo de aplicaciones modernas.

Permite ejecutar varios servicios al mismo tiempo con un solo comando, lo que facilita la configuración de entornos complejos.
También simplifica la administración de contenedores, reduce errores de configuración y mejora la reproducibilidad del sistema.

Además, Docker Compose es especialmente útil en arquitecturas basadas en microservicios, donde se requiere coordinar múltiples servicios como bases de datos, servidores web y aplicaciones backend.

Comandos utilizados en la práctica

docker compose up -d

docker compose ps

docker compose down

docker logs db-postgres

docker inspect db-postgres

Si quieres, lo siguiente te lo puedo preparar también:

- el **docker-compose.yml exacto para tu proyecto real**
- o el **paso a paso para ejecutar esto en tu computador**